UNIT - III : DICTIONARIES, TUPLES AND SETS

I. Dictionaries

* A DICTIONARY (dict) is an unordered-by-index (insertion ordered from Python 3.7), MUTABLE collection of KEY : VALUE pairs inside curly braces { }. Keys must be UNIQUE and IMMUTABLE (int, str, tuple); values can be any type and may repeat. Access is by key (hash table - very fast lookup).

Creating a dictionary

```
d1 = {}                                    # empty dict
d2 = {"name": "Ravi", "age": 20, "city": "Hyd"}
d3 = dict(a=1, b=2)                        # using dict()
d4 = dict([("x", 10), ("y", 20)])          # from list of tuples
d5 = dict.fromkeys(["a", "b", "c"], 0)     # {'a':0,'b':0,'c':0}
sq = {x: x*x for x in range(1, 5)}         # comprehension
print(sq)                                  # {1:1, 2:4, 3:9, 4:16}
```

Accessing and modifying key:value pairs

```
d = {"name": "Ravi", "age": 20}
print(d["name"])              # Ravi   (KeyError if missing)
print(d.get("city"))          # None   (no error)
print(d.get("city", "NA"))    # NA     (default value)
d["age"] = 21                 # modify existing
d["city"] = "Hyd"             # add new pair
d.update({"age": 22, "phone": 99})   # update / add many
print(d)
```

```
for k in d:                     # keys
    print(k, end=" ")
for k, v in d.items():          # key and value
    print(k, "->", v)
print("age" in d)               # True  (membership tests keys)
```

Built-in functions used on dictionaries

Function	Result for d = {'a':3,'b':1,'c':2}
len(d)	3 (number of pairs)
max(d), min(d)	'c', 'a' (compares KEYS)
sorted(d)	['a', 'b', 'c'] (sorted keys)
sum(d.values())	6
list(d), tuple(d)	['a','b','c'], ('a','b','c')
any(d), all(d)	True, True (on keys)
str(d), type(d)	string form, <class 'dict'>

Dictionary methods

Method	Action
keys()	View of all keys
values()	View of all values
items()	View of (key, value) tuples
get(k, default)	Value of key k, or default / None
update(d2)	Add / change pairs from d2
pop(k)	Remove key k and return its value
popitem()	Remove and return last inserted pair
setdefault(k, v)	Return value if k exists, else insert k:v
copy()	Shallow copy
clear()	Remove all pairs
fromkeys(seq, v)	New dict with keys from seq

```
d = {"a": 1, "b": 2, "c": 3}
print(d.keys())          # dict_keys(['a', 'b', 'c'])
print(d.values())        # dict_values([1, 2, 3])
print(d.items())         # dict_items([('a',1),('b',2),('c',3)])
print(d.pop("b"))        # 2
print(d.popitem())       # ('c', 3)
d.setdefault("z", 26)
print(d)                 # {'a': 1, 'z': 26}
```

del statement

```
d = {"a": 1, "b": 2, "c": 3}
del d["b"]           # removes the key 'b'  -> {'a':1,'c':3}
d.clear()            # {} - empties the dictionary
del d                # deletes the dictionary object itself
```

Nested dictionary

```
students = {
    101: {"name": "Anu", "marks": 85},
    102: {"name": "Raj", "marks": 72},
}
print(students[101]["name"])      # Anu
students[102]["marks"] = 80
for roll, info in students.items():
    print(roll, info["name"], info["marks"])
```

Common dictionary programs

```
# 1. word / character frequency
text = "banana"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)                 # {'b': 1, 'a': 3, 'n': 2}

# 2. sort by key and by value
d = {"x": 30, "a": 10, "m": 20}
print(sorted(d.items()))                          # by key
print(sorted(d.items(), key=lambda p: p[1]))      # by value
print(dict(sorted(d.items(), key=lambda p: p[1], reverse=True)))

# 3. merge two dictionaries
d1 = {"a": 1}; d2 = {"b": 2, "a": 9}
print({**d1, **d2})         # {'a': 9, 'b': 2}   (d1 | d2 in 3.9+)

# 4. invert keys and values
inv = {v: k for k, v in {"a": 1, "b": 2}.items()}   # {1:'a', 2:'b'}
```

Operation	Time	Note
d[key], get()	O(1) average	hash lookup
d[key] = v	O(1) average	insert / update
key in d	O(1) average	fast membership
key in list	O(n)	slow - scans the list

Keys must be immutable & unique (list as key -> TypeError: unhashable). Duplicate key in the literal keeps the LAST value.

2. Tuples

* A TUPLE is an ordered, IMMUTABLE sequence of items in round brackets ( ). Once created, items cannot be added, changed or removed. Allows duplicates and mixed types. Faster and uses less memory than a list; can be used as dictionary key; used for fixed data (days of week, coordinates).

Creating tuples

```
t1 = ()                         # empty tuple
t2 = (10, 20, 30)
t3 = 5, "abc", 2.5              # packing (brackets optional)
t4 = (7,)                       # single item needs a comma!
t5 = (7)                        # NOT a tuple - just int 7
a, b, c = t2                    # unpacking
print(type(t4), type(t5))       # tuple, int
```

Basic tuple operations

```
x = (1, 2, 3); y = (4, 5)
print(x + y)         # (1, 2, 3, 4, 5)  concatenation
print(x * 2)         # (1, 2, 3, 1, 2, 3)
print(2 in x)        # True
print(len(x))        # 3
for i in x: print(i, end=" ")
x[0] = 9             # TypeError: immutable
```

tuple() function, indexing and slicing

```
print(tuple([1, 2, 3]))       # (1, 2, 3)  list -> tuple
print(tuple("hi"))            # ('h', 'i')
print(tuple(range(4)))        # (0, 1, 2, 3)
t = (10, 20, 30, 40, 50)
print(t[0], t[-1])            # 10 50
print(t[1:4])                 # (20, 30, 40)
print(t[::-1])                # (50, 40, 30, 20, 10)
```

Built-in functions and methods on tuples

```
t = (4, 1, 9, 1)
print(len(t), max(t), min(t), sum(t))    # 4 9 1 15
print(sorted(t))              # [1, 1, 4, 9]   (returns LIST)
print(t.count(1))             # 2
print(t.index(9))             # 2
```

* Only 2 tuple methods: count() and index().

```
# nested tuple, swap and returning many values
t = ((1, 2), (3, 4), "end")
print(t[1][0])                  # 3
a, b = 5, 9
a, b = b, a                     # swap using tuple packing
def stats(nums):
    return min(nums), max(nums), sum(nums)
lo, hi, tot = stats([4, 8, 1])    # tuple unpacking
first, *rest = (1, 2, 3, 4)       # first=1, rest=[2,3,4]
```

* Tuples can contain mutable objects: t = (1, [2, 3]); t[1].append(4) works -> (1, [2, 3, 4]); but t[1] = [9] fails. del t deletes the whole tuple (cannot delete single items).

Relation between tuples and lists

Tuple	List
Immutable	Mutable
( ) round brackets	[ ] square brackets
Faster, less memory	Slower, more memory
Only count(), index() methods	Many methods (append, sort ...)
Can be a dictionary key	Cannot be a dictionary key
Fixed / heterogeneous data	Variable / homogeneous data

```
L = list((1, 2, 3))             # tuple -> list (to modify)
L.append(4)
T = tuple(L)                    # list -> tuple
print(L, T)                     # [1,2,3,4] (1,2,3,4)
```

Relation between tuples and dictionaries

* dict.items() returns (key, value) TUPLES; a list of 2-item tuples can be converted to a dict with dict(); tuples can be used as KEYS of a dictionary (because immutable).

```
d = {"a": 1, "b": 2}
lst = list(d.items())              # [('a', 1), ('b', 2)]
d2 = dict(lst)                     # back to dict
loc = {(17.38, 78.48): "Hyderabad", (28.6, 77.2): "Delhi"}
print(loc[(28.6, 77.2)])           # Delhi
```

zip() function

* zip() combines items of two or more iterables position by position into TUPLES; stops at the shortest. Result is an iterator - wrap in list(), dict(). zip(*pairs) unzips.

```
names = ["Anu", "Raj", "Sam"]
marks = [85, 72, 90]
z = list(zip(names, marks))
print(z)              # [('Anu',85), ('Raj',72), ('Sam',90)]
print(dict(zip(names, marks)))    # {'Anu':85,'Raj':72,'Sam':90}
n2, m2 = zip(*z)      # unzip
for n, m in zip(names, marks):
    print(n, m)
```

3. Sets

* A SET is an UNORDERED collection of UNIQUE items in curly braces { }. Mutable (add / remove) but elements must be immutable; no

indexing or slicing; duplicates removed automatically. Used for membership tests and mathematical set operations.

```
s1 = {1, 2, 3, 3, 2}          # duplicates removed
print(s1)                     # {1, 2, 3}
s2 = set([4, 5, 5, 6])        # {4, 5, 6}
s3 = set("hello")             # {'h','e','l','o'}
empty = set()                 # NOT {} (that is an empty dict)
print(len(s1), 2 in s1)       # 3 True
print(list(set([1, 1, 2, 3, 3])))   # remove duplicates
```

Set methods

Method	Action
add(x)	Add one element
update(iter)	Add many elements
remove(x)	Delete x (KeyError if absent)
discard(x)	Delete x (no error if absent)
pop()	Remove and return an arbitrary element
clear(), copy()	Empty / shallow copy
union(s2) |	All elements of both
intersection(s2) &	Common elements
difference(s2) -	In s1 but not in s2
symmetric_difference(s2) ^	In either, not both
issubset(), issuperset(), isdisjoint()	Relationship tests (True / False)

```
A = {1, 2, 3, 4}; B = {3, 4, 5, 6}
print(A | B)        # {1, 2, 3, 4, 5, 6}   union
print(A & B)        # {3, 4}               intersection
print(A - B)        # {1, 2}               difference
print(A ^ B)        # {1, 2, 5, 6}         symmetric difference
print({3, 4} <= A)  # True                 subset
A.add(10); A.discard(1); A.remove(2)
print(A)            # {3, 4, 10}
```

```
# practical uses of sets
nums = [1, 2, 2, 3, 4, 4, 5]
print(list(set(nums)))                  # remove duplicates
a = set("python"); b = set("typhoon")
print(a & b)                            # common letters
print(a - b)                            # letters only in a
print(len(set("mississippi")))          # 4 unique letters
print(set(nums) == {1, 2, 3, 4, 5})     # True (order ignored)
s = {1, 2}
s.update([3, 4], {5})                   # add many items
print(sorted(s, reverse=True))          # [5, 4, 3, 2, 1]
```

Set operation	Operator	Method	Venn meaning
Union	A | B	A.union(B)	everything in A or B
Intersection	A & B	A.intersection(B)	only the overlap
Difference	A - B	A.difference(B)	A without the overlap
Symmetric diff.	A ^ B	A.symmetric_difference(B)	both except overlap

Frozenset

* A FROZENSET is an IMMUTABLE set: created with frozenset(iterable); supports all non-modifying set operations (union, intersection, difference, issubset) but NOT add / remove. Because it is hashable, a frozenset can be a dictionary key or an element of another set.

```
f = frozenset([1, 2, 3, 2])
print(f)                          # frozenset({1, 2, 3})
print(f | frozenset([5]))         # frozenset({1, 2, 3, 5})
f.add(4)                          # AttributeError: no attribute 'add'
d = {frozenset([1, 2]): "pair"}   # allowed as a key
```

	List	Tuple	Set	Dictionary
Brackets	[ ]	( )	{ }	{k: v}
Ordered	Yes	Yes	No	Insertion order
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	Yes	Yes	No	By key

4. Sample Experiments (13 - 17)

Exp 13: Create tuples (name, age, address, college) and concatenate

```
m1 = ("Anu", 20, "Hyderabad", "JNTU")
m2 = ("Raj", 21, "Warangal", "NIT")
result = m1 + m2
print("Member 1:", m1)
print("Member 2:", m2)
print("Concatenated tuple:", result)
# ('Anu', 20, 'Hyderabad', 'JNTU', 'Raj', 21, 'Warangal', 'NIT')
```

Exp 14: Count the vowels in a string (no control flow)

* No if / for / while allowed - use string method count() and sum(), or len() with a set / map.

```
s = input("Enter a string: ").lower()
vowels = s.count("a") + s.count("e") + s.count("i") + \
         s.count("o") + s.count("u")
print("Number of vowels:", vowels)
# Input: Python Programming -> Number of vowels: 4
# another way: print(sum(map(s.count, "aeiou")))
```

Exp 15: Check if a given key exists in a dictionary

```
d = {"name": "Ravi", "age": 20, "city": "Hyd"}
key = input("Enter key to search: ")
if key in d:
    print("Key exists, value =", d[key])
else:
    print("Key does not exist")
# Input: age -> Key exists, value = 20
```

Exp 16: Add a new key-value pair to an existing dictionary

```
d = {"a": 1, "b": 2}
print("Before:", d)
k = input("New key: ")
v = int(input("New value: "))
d[k] = v                          # or d.update({k: v})
print("After :", d)
# Input: c 3 -> After : {'a': 1, 'b': 2, 'c': 3}
```

Exp 17: Sum all the items in a dictionary

```
d = {"a": 100, "b": 200, "c": 300}
total = sum(d.values())
print("Sum of all items =", total)       # 600
# using loop
t = 0
for v in d.values():
    t += v
print(t)
```

Output prediction practice

```
print(type({}), type(()), type({1}), type((1,)), type((1)))
# <class 'dict'> <class 'tuple'> <class 'set'> <class 'tuple'> <class
d = {1: "a", 2: "b", 1: "c"}
print(d, len(d))                  # {1: 'c', 2: 'b'} 2
print({1, 2, 3} == {3, 2, 1})     # True
print(list(zip("ab", [1, 2, 3])))    # [('a', 1), ('b', 2)]
print(max({"x": 5, "y": 9}, key=lambda k: {"x": 5, "y": 9}[k]))  # y
```

Important questions

* 1. Explain dictionary creation, accessing and methods with examples.
* 2. Differentiate list, tuple, set and dictionary.
* 3. What is zip()? Explain with an example. What is a frozenset?
* 4. Explain set operations - union, intersection, difference, symmetric difference.

Revision: dict = key:value, keys unique and immutable. Tuple = immutable, single item needs comma. Set = unique, unordered, set() for empty. Frozenset = immutable set. zip() pairs items.